

Implementation Plan and Expected Outputs

Detecting Blockchain-Anchored C2 in IoT Botnets — shown side-by-side with the offline-RL sepsis paper used as the reference template

0. The short answer: yes, ours is an equally real, implementable project

The sepsis paper is a finished, empirical study: it took real data, ran several algorithms against each other, and produced concrete outputs — convergence curves, comparison tables, SHAP plots, and cross-cohort results. Our project follows the exact same pattern. The one structural difference is the data source: the sepsis authors downloaded ready-made labelled datasets (MIMIC-III, eICU); we generate our labelled traffic in an isolated testbed and supplement it with public IoT datasets. Everything after data collection — feature engineering, benchmarking multiple detectors, imbalance handling, ablation, SHAP explainability, cross-dataset generalization — is the same kind of work and produces the same kinds of outputs. This document shows exactly how we implement each stage and what each stage produces.

1. How our project lines up with the reference paper

Sepsis paper element	Our project equivalent	Same technique?
Data: MIMIC-III + eICU (downloaded, labelled)	Testbed-generated PCAP traces + public IoT datasets (IoT-23, Bot-IoT, N-BalIoT)	Yes — supervised labelled data
Action imbalance 384:1, capped inverse-frequency weighting	Benign-vs-malicious flow imbalance; class weights / SMOTE	Yes — imbalance correction
Benchmark 3 algorithms: FQI, DQN, CQL	Benchmark detectors: heuristic baseline, Random Forest, XGBoost, LSTM	Yes — multi-model comparison
Reward shaping (domain-grounded)	Feature engineering (resolution + relay indicators)	Yes — domain-grounded inputs
Multi-seed robustness (5 seeds)	Multi-seed training, mean $\pm$ std	Yes — identical protocol
Ablation study	Ablation: drop each indicator group	Yes — identical protocol
SHAP explainability (platelet, temp, creatinine)	SHAP on flow features (ENS-query rate, UPnP mapping, beacon interval)	Yes — same library, same idea
Cross-cohort generalization (MIMIC $\rightarrow$ eICU)	Cross-dataset generalization (train family A $\rightarrow$ test family B)	Yes — identical protocol
Off-policy value estimates	Held-out and false-positive analysis on benign traffic	Analogous — rigorous evaluation

The takeaway: the reference paper is essentially a supervised/RL benchmarking study with explainability. Our project is a supervised detection benchmarking study with explainability. The scaffolding is the same; only the domain and the data-collection step differ.

2. How we implement it (stage by stage)

Stage 1 — Data and trace generation

- Build an isolated virtual lab (VirtualBox/QEMU or GNS3) with no route to the public internet: emulated IoT devices (OpenWrt images), a local stand-in ENS/SNS resolver, and a controlled relay node.
- Capture all traffic with tcpdump / Wireshark into PCAP files while reproducing the observable behaviours: blockchain-name resolution query, HTTP 'server-list' pull, UPnP port-mapping, and long-lived bidirectional relay flows.
- Add public datasets for realism and a negative class: IoT-23, Bot-IoT, N-BalIoT, and CIC-IDS for benign + known-botnet traffic (Mirai-family), so the model sees real IoT noise.
- Static-only analysis of public malware samples / published IOCs to confirm which indicators to engineer (no execution of live malware, no live C2).

Output of this stage: a labelled corpus of PCAP files tagged benign / suspicious-resolution / relay, plus a documented IOC catalogue. (Parallel to the sepsis 'Datasets and MDP Formulation' section.)

Stage 2 — Feature extraction

- Convert PCAPs to flow records using Zeek and/or CICFlowMeter; use Scapy for custom packet-level features.
- Engineer the discriminating features: rate of queries to ENS/SNS gateways or blockchain RPC endpoints; presence of an HTTP server-list pull after resolution; UPnP AddPortMapping events; bidirectional flow duration and fan-out; beacon periodicity; RC4-encrypted string signatures; Telnet/SSH login-burst counts.

Output: a clean feature matrix (rows = flows/devices, columns = engineered features, plus a label). This is your equivalent of their preprocessed MDP state features.

Stage 3 — Detection models (the benchmark)

- Heuristic baseline: threshold rules on the strongest indicators — transparent and explainable (your 'FQI-like' simple baseline).
- Random Forest and XGBoost: interpretable, low-compute tabular classifiers (your main 'DQN-like' workhorses).
- Optional LSTM / 1D-CNN on flow sequences to capture beaconing rhythm (your 'CQL-like' more powerful model).
- Handle class imbalance with class weights or SMOTE — directly parallel to their capped inverse-frequency weighting.

Output: three-to-four trained detectors, ready for head-to-head comparison.

Stage 4 — Evaluation, ablation, and explainability

- Metrics: Precision, Recall, F1, ROC-AUC, and a dedicated false-positive rate on benign traffic.
- Multi-seed robustness: repeat training over 5 seeds, report mean $\pm$ standard deviation.
- Ablation: remove each feature group (resolution / relay / infection / payload) and measure the drop — proves which indicators matter.
- Cross-dataset generalization: train on one botnet family/dataset, test on another.

- SHAP explainability: identify which features drive detections — the direct analogue of their SHAP analysis naming platelet/temperature/creatinine.

Output: the full results set (tables + figures) listed in Section 3 below.

Stage 5 — Analysis, artifact release, and paper

- Interpret results, discuss limitations and deployment, release the labelled dataset + code with a reproducibility README, and write the paper in the reference paper's structure.

3. Concrete outputs (what the project actually produces)

Every output below is directly analogous to a figure or table in the sepsis paper, so a reviewer will recognise the same level of rigor.

#	Output (figure/table)	Sepsis-paper analogue
1	System/pipeline architecture diagram	Their Figure 1 (framework overview)
2	Dataset composition table + class-imbalance ratio	Their action-imbalance (384:1) reporting
3	Model training/convergence curves	Their DQN convergence plots
4	Model comparison table (heuristic vs RF vs XGB vs LSTM): P/R/F1/AUC	Their FQI-DQN-CQL comparison
5	Multi-seed robustness table (mean $\pm$ std)	Their 5-seed robustness
6	Ablation table (contribution of each feature group)	Their ablation study
7	ROC and Precision-Recall curves	Their off-policy value curves
8	Confusion matrices + false-positive analysis	Their clinical-override analysis
9	SHAP summary plot (top detection-driving features)	Their SHAP feature-importance
10	Cross-dataset generalization matrix	Their MIMIC→eICU cross-cohort table
11	Released labelled dataset + reproducible code	Their reproducibility artifact

4. Tools and tech stack

Purpose	Tools
Testbed / emulation	VirtualBox or QEMU, GNS3, OpenWrt device images
Traffic capture	tcpdump, Wireshark
Feature extraction	Zeek, CICFlowMeter, Scapy, pandas
Modelling	scikit-learn (Random Forest), XGBoost, PyTorch (optional LSTM)
Imbalance handling	imbalanced-learn (SMOTE), class weights
Explainability	SHAP
Plots / reporting	Matplotlib, Seaborn
Public datasets	IoT-23, Bot-IoT, N-BaIoT, CIC-IDS (benign + known

	botnet)
--	---------

5. The one honest caveat

The sepsis team's biggest advantage was a large, ready-made, labelled dataset. We do not have that for a July-2026 botnet — so our Stage 1 (testbed trace generation) is real work and the main risk to matching this paper's depth. **Mitigation:** lean on established public IoT/botnet datasets for volume and the negative class, use the testbed to synthesise the specific blockchain-C2 and relay behaviours, and be explicit in the paper that the blockchain-C2 traces are synthesised in a controlled lab. Reviewers accept synthetic/testbed data when the generation process is documented and reproducible — which is itself a contribution.

Bottom line: the project is fully implementable, produces the same class of concrete outputs as the reference paper, and fits the same section structure. The work that earns the paper is the empirical depth — building the dataset and running the full benchmark + ablation + SHAP + generalization suite.